Роботу виконано на гілці **main**, на основі версії застосунку з практикуму 8. Запити до JSONPlaceholder з практикуму 9 сюди не потрапили: вони лишаються в окремій гілці lab9-fetch-api. Тому в компонент передається масив записів настрою з практикуму 6. Варіант 3: компонент MoodEntry, props date, mood, note. Похідне значення emoji обчислюється з mood, а не зберігається окремо.

```
/*
 * Lab 10. Framework: Vue 3 (global CDN build, no Node.js or bundler).
 * Why Vue over React: Vue needs a single <script> tag and no Babel, its templates are plain
HTML
 * with directives (v-for, @click), so the existing card markup moves over almost unchanged,
 * and its reactivity on direct data changes fits well with the rest of the vanilla code
 * (the form and the summary stay on the DOM API and just pass data into Vue state).
 */

class MoodCard {
    constructor(moodLevel, note, dateTime = new Date()){
        this.id = nextId();
        this.DateTime = dateTime;
        this.moodLevel = moodLevel;
        this.note = note;
    }
}

// returns a unique id for a mood record
let idCounter = 0;
function nextId(){
    idCounter++;
    return idCounter;
}

const generalTips = [
    "A regular sleep routine can help support a steadier mood.",
    "Drink enough water: even mild dehydration can affect how you feel.",
    "A 15-minute walk outside can lift your mood.",
    "Write down one small thing you are grateful for today.",
    "Take short breaks from screens during study sessions.",
    "Talking to a friend can make a heavy day feel lighter.",
];

// returns the date n days before today (midnight)
function daysAgo(n){
    const date = new Date();
    date.setHours(0, 0, 0, 0);
    date.setDate(date.getDate() - n);
    return date;
}

// returns the starting list of mood records
function initialMoodHistory(){
    return [
        new MoodCard(1, "very bad, nothing worked today", daysAgo(7)),
        new MoodCard(2, "tired after classes", daysAgo(6)),
        new MoodCard(3, "just a regular day", daysAgo(5)),
        new MoodCard(4, "finished the lab on time", daysAgo(4)),
        new MoodCard(5, "great day, met friends", daysAgo(3)),
        new MoodCard(3, "okay, a bit sleepy", daysAgo(2)),
```

```
        new MoodCard(4, "good walk in the evening", daysAgo(1)),
    ];
}

const balanceTips = {
    low: [
        "Your mood has been low lately. Try to rest, eat well, and talk to someone you
trust.",
        "A tough stretch is still just a stretch. Pick one small thing today that makes you
feel a bit better.",
    ],
    mid: [
        "Your mood balance is steady. A short walk or a call with a friend could tip it
towards the good side.",
        "You are doing okay. Try adding one small thing you enjoy to each day.",
    ],
    high: [
        "Your mood balance looks great. Note what is working so you can repeat it.",
        "Good streak! Keep up the habits that got you here, like sleep, movement and time
with friends.",
    ],
};

// updates the summary texts based on the mood records
function updStats(articles) {
    const sum1 = document.querySelector('p#sumtxt1');

    if (sum1) {
        sum1.textContent = genSumTxt1(articles);
    }

    const sum2 = document.querySelector('p#sumtxt2');
    if (sum2) {
        sum2.textContent = genSumTxt2(articles);
    }

    const sum3 = document.querySelector('p#avg-mood');
    if (sum3) {
        sum3.textContent = genSumTxt3(articles);
    }

    const sum4 = document.querySelector('p#sumtxt4');
    if (sum4) {
        sum4.textContent = genSumTxt4();
    }
}

// counts records with the given mood level among the first `scope` records
function countMood(searchedMoodLevel, scope, articles){
    if (scope > articles.length){
        return -1;
    }

    let count = 0;
    for (let i = 0; i < scope; i++){
        if(articles[i].moodLevel === searchedMoodLevel){
            count++;
        }
    }
    return count;
```

```
}

// mood percentages for the last 7 records
function genSumTxt1(articles){
    const n = Math.min(7, articles.length);
    const pct = level => Math.round(countMood(level, n, articles) / n * 100);
    return `Your recent moods were ${pct(5)}% very good, ${pct(4)}% good, ${pct(3)}% okay,
${pct(2)}% bad, and ${pct(1)}% very bad.`
}

// overall tone of the last 7 records
function genSumTxt2(articles){
    const recent = articles.slice(0, 7);
    const positive = recent.filter(a => a.moodLevel >= 4).length;
    const negative = recent.filter(a => a.moodLevel <= 2).length;
    let tone;
    if (positive > negative && negative <= recent.length / 4) {
        tone = "positive or okay";
    } else if (negative > positive) {
        tone = "low";
    } else {
        tone = "mixed";
    }
    return `Most of your recent moods were ${tone}. Keep tracking them to notice what
affects your wellbeing.`;
}

// gets random item in list
const randomItem = list => list[Math.floor(Math.random() * list.length)];

// tip based on average mood of the last 7 entries
function genSumTxt3(articles){
    const avg = average(articles.slice(0, 7));
    const bucket = avg < 2.5 ? 'low' : avg < 3.5 ? 'mid' : 'high';
    return randomItem(balanceTips[bucket]);
}

// random general wellbeing tip
function genSumTxt4(){
    return randomItem(generalTips);
}

// finds average moodLevel in array of MoodCard
function average  (moods_array) {
    let avarage_val = 0;
    for (let mood_date of moods_array) {
        avarage_val += mood_date.moodLevel;
    }
    return avarage_val/(moods_array.length);
}

// logs whether the week was good or bad
function logWeek(articles){
    if (average(articles) >= 3){
        console.log("Good week");
    }
    else {
        console.log("Bad week");
    }
}
```

```
// card for a single mood record; emoji, card class and date are computed from props
const MoodEntry = {
    props: {
        date: { type: Date, required: true },
        mood: { type: Number, required: true, validator: value => value >= 1 && value <= 5
},
        note: { type: String, default: '' },
    },
    emits: ['cycle-mood'],
    computed: {
        emoji() {
            return ['😞', '😟', '😐', '🙂', '😄'][this.mood - 1];
        },
        label() {
            return ['very_bad', 'bad', 'okay', 'good', 'very_good'][this.mood - 1];
        },
        formattedDate() {
            return this.date.toLocaleDateString('uk-UA');
        },
    },
    template: `
        <article :class="'card_' + label" :data-mood="mood">
            <div class="entry-head">
                <h3>{{ formattedDate }}</h3>
                <button type="button" class="mood-emoji"
                        :title="'Mood ' + mood + '/5 - click to change'"
                        :aria-label="'Mood ' + mood + ' of 5, change mood'"
                        @click="$emit('cycle-mood')">{{ emoji }}</button>
            </div>
            <p>{{ note }}</p>
        </article>`,
};

// mood history section: holds the records and renders them as MoodEntry cards
const historyApp = Vue.createApp({
    components: { MoodEntry },
    data() {
        return {
            entries: initialMoodHistory(),
        };
    },
    computed: {
        // records sorted by date, newest first
        sortedEntries() {
            return [...this.entries].sort((a, b) => b.DateTime - a.DateTime);
        },
    },
    watch: {
        // refreshes the summary whenever records are added or changed
        entries: {
            handler() {
                updStats(this.sortedEntries);
                logWeek(this.entries);
            },
            deep: true,
            immediate: true,
        },
    },
    methods: {
```

```
        // adds a new record to the list
        addEntry(card) {
            this.entries.push(card);
        },
        // switches the record's mood to the next level (5 wraps to 1)
        cycleMood(id) {
            const entry = this.entries.find(e => e.id === id);
            if (entry) {
                entry.moodLevel = (entry.moodLevel % 5) + 1;
            }
        },
    },
    template: `
        <h2>Your mood history</h2>
        <div class="cards">
            <MoodEntry
                v-for="entry in sortedEntries"
                :key="entry.id"
                :date="entry.DateTime"
                :mood="entry.moodLevel"
                :note="entry.note"
                @cycle-mood="cycleMood(entry.id)" />
        </div>`,
});

const historyVm = historyApp.mount('#history');


const form = document.querySelector('#mood-form');
const dateInput = document.querySelector('#mood-date');
const moodInput = document.querySelector('#mood-level');
const noteInput = document.querySelector('#mood-note');
const noteCounter = document.querySelector('#note-counter');

// updates the counter of characters left in the note
function updNoteCounter(){
    const left = noteInput.maxLength - noteInput.value.length;
    noteCounter.textContent = `${left} characters left`;
}

// extra mood validation: value must be between 1 and 5
moodInput.addEventListener('input', () => {
    const value = Number(moodInput.value);
    if (moodInput.value !== '' && (value < 1 || value > 5)) {
        moodInput.setCustomValidity('Mood must be between 1 and 5');
    } else {
        moodInput.setCustomValidity('');
    }
});

noteInput.addEventListener('input', updNoteCounter);

// form submit: adds a new record without reloading the page
form.addEventListener('submit', event => {
    event.preventDefault();

    const date = new Date(`${dateInput.value}T00:00`);
    const moodLevel = Number(moodInput.value);
    const note = noteInput.value.trim();
```

```
    historyVm.addEntry(new MoodCard(moodLevel, note, date));

    form.reset();
    moodInput.setCustomValidity('');
    updNoteCounter();
});
```

```
<section class="history" id="history"></section>
...

<script defer src="https://unpkg.com/vue@3.5.13/dist/vue.global.js"></script>
<script defer src="js/app.js"></script>
```

№1

Обрано Vue 3. Він підключається одним тегом script без Babel, а його шаблони - це звичайний HTML з директивами, тож розмітку карток вдалося перенести майже без змін. Крім того, Vue сам відстежує пряму зміну даних, тому його зручно поєднати з рештою ванільного коду (форма і блок підсумку). Обґрунтування вибору записано коментарем на початку app.js.

№2

У index.html перед app.js підключено глобальну збірку vue@3.5.13 з unpkg. Версію зафіксовано, щоб оновлення бібліотеки не зламало сторінку. Обидва скрипти мають атрибут defer, тому Vue завантажується раніше за app.js.

№3

Масив записів більше не зберігається в глобальній змінній moodHistory. Функція initialMoodHistory() повертає його в data() кореневого компонента як entries, і там він стає реактивним станом. Функції підсумку countMood і logWeek тепер отримують масив параметром.

№4, 5



Компонент MoodEntry отримує дані лише через props: date (Date), mood (Number, з перевіркою 1-5) і note (String). Emoji, CSS-клас card_* і відформатована дата - це computed-значення від props. Кореневий компонент рендерить картки через v-for по sortedEntries (computed, нові записи йдуть першими) з :key="entry.id". Для ключа в клас MoodCard додано поле id.

№6

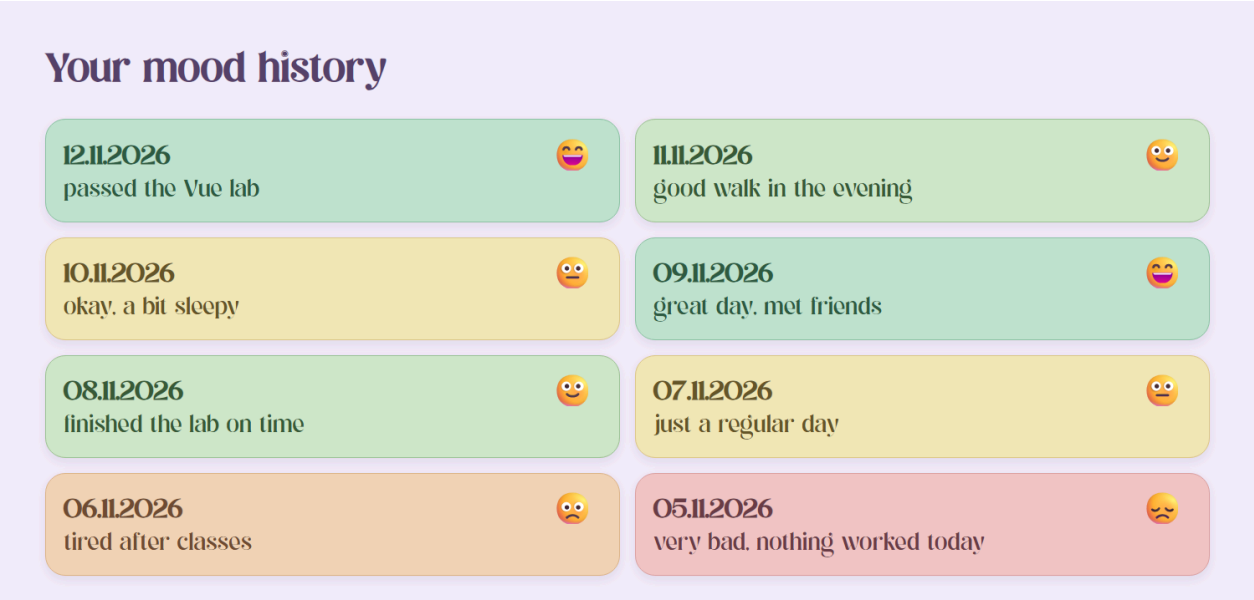


Клік по emoji викликає $emit('cycle-mood'). Батьківський компонент у методі cycleMood(id) змінює moodLevel запису на наступний (після 5 знову 1). На скріншоті - запис 11.11.2026 після кліку: настрій змінився з 4 (🙂) на 5 (😄), і разом з ним змінився колір картки.

№7

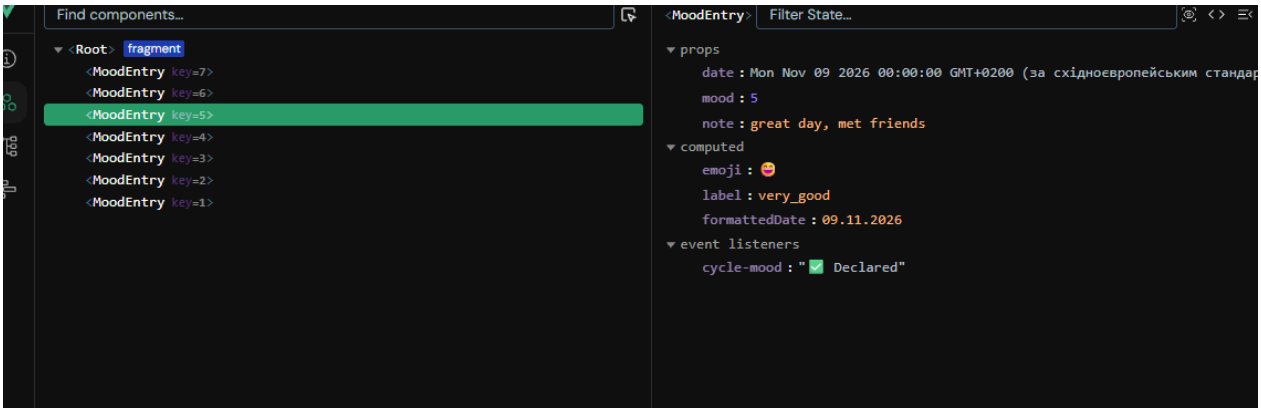
Функції renderArticles, sortByDate і moodToLabel видалено: їх замінено компонентом MoodEntry, директивою v-for і computed-властивостями. Секція #history у розмітці тепер порожня, її вміст повністю створює Vue.

№8



Форма лишилась на DOM API, але тепер передає новий запис у стан Vue через historyVm.addEntry(...). Список перемальовується сам, новий запис стає першим. Блок підсумку оновлюється через deep watch на entries - як після додавання запису, так і після кліку по emoji.

№9



У Vue DevTools дерево компонентів - Root з сімома дочірніми MoodEntry, по одному на кожен запис; key кожного компонента збігається з id запису. Для вибраного MoodEntry (key=5) видно props date, mood і note, обчислені з них computed emoji, label і formattedDate, а також оголошений слухач події cycle-mood.

№10

Коментар з обраним фреймворком і причиною вибору є на початку app.js. Над функціями та методами залишено короткі коментарі англійською про те, що вони роблять.